exercise 8.

Problem 1.

1. The reversal $rev(G)$ is the graph obtained by reversing the direction of every edge in G. That is, if $G=(V,E)$, the ~~rev~~ $rev(G)=(V,E')$, where $E'=\{(v,u):(u,v)\in E\}$.

① Create a new adjacency list representation revadj with V vertices and no edges. Time: $O(V)$

② For each vertex u in G, iterate over all edges (u,v) in $adj[u]$. For each such edge, add u to $revadj[v]$. Time: $O(V+E)$, since we visit each vertex and each edge exactly once.

③ Return revadj as the adjacency list of $rev(G)$.

```
REG(G):
    Greate G' with vertex set V(G), empty edge set
    for each vertex u in V(G):
        for each v in Adj[u]:
            Add u to Adj'[v]        # add edge (v,u) to G'
    return G'
```

Time Analysis: Greating the vertex set takes $O(V)$. We iterate over every edge exactly once, so the loop takes $O(E)$. Total: $O(V+E)$ ※

2. proof:

Recall that $scc(G)$ is defined by merging every strongly connected component $CSCC$ of G into a single node. Two nodes C_1, C_2 in $scc(G)$ are connected by an edge $C_1 \to C_2$ if and only if there exists $u\in C_1$ and $v\in C_2$ such that $(u,v)\in E(G)$.

Assume for contradiction that $scc(G)$ contains a cycle $C_1 \to C_2 \to \dots \to C_k \to C_1$ $(k\geq 2)$. Then for any vertex $a\in C_1$ and $b\in C_2$, we can reach from a to some vertex in C_2 (through the inter-component edge), then reach b (within C_2, since it is strongly connected). Continuing along the cycle, we can reach from b back to a vertex in C_1,

and then back to a.

This means all vertices in $C_1 \cup C_2 \cup \cdots \cup C_k$ are mutually reachable - they should all belong to the same SCC. This contradicts the fact that $C_1, C_2, \ldots, C_k$ are distinct SCCs. Therefore $scc(G)$ is acyclic (a DAG).

3. proof:

① Same SCCs.

vertices u and v are in the same SCC of G if and only if there exist paths $u \to v$ and $v \to u$ in G. Reversing all edges, this is equivalent to paths $v \to u$ and $u \to v$ existing in $rev(G)$. Thus u and v are mutually reachable in G if and only if they are mutually reachable in $rev(G)$. Hence G and $rev(G)$ have exactly the same SCCs.

So, both $scc(rev(G))$ and $rev(scc(G))$ have the same vertex set (each vertex = one SCC)

② Same edges

Let C_1 and C_2 be two distinct SCCs. Then:

Edge (C_1, C_2) in $rev(scc(G)) \Leftrightarrow$ edge (C_2, C_1) in $scc(G)$

$\Leftrightarrow \exists$ edge (u, v) in G with $u \in C_2$, $v \in C_1 \Leftrightarrow \exists$ edge (v, u) in $rev(G)$ with $v \in C_1$, $u \in C_2 \Leftrightarrow$ edge (C_1, C_2) in $scc(rev(G))$.

Since both graphs have the same nodes and the same edges, $scc(rev(G)) = rev(scc(G))$. ※

4. proof:

$(\Rightarrow)$:

Suppose u can reach v in G via a path $u = w_0 \to w_1 \to \ldots \to w_k = v$. Consider the sequence of SCCs: $S(w_0), S(w_1), \ldots, S(w_k)$. For consecutive vertices w_i and w_{i+1}, the edge (w_i, w_{i+1}) exists in G. If $S(w_i) = S(w_{i+1})$, they are in the same SCC. If $S(w_i) \neq S(w_{i+1})$, then there is an edge $S(w_i) \to S(w_{i+1})$ in $scc(G)$. By removing repeated SCCs, we obtain a path from $S(u)$ to $S(v)$ in $scc(G)$.

$(\Leftarrow)$

Suppose $S(u)$ can reach $S(v)$ in $scc(G)$ via a path $S(u) = C_0 \to C_1 \to \ldots \to C_m = S(v)$. Since $C_0 \to C_1$ is an edge in $scc(G)$, there exist $a_0 \in C_0$ and $b_1 \in C_1$ such that $(a_0, b_1) \in E(G)$. Similarly for each edge $C_i \to C_{i+1}$, there exist $a_i \in C_i$, $b_{i+1} \in C_{i+1}$ with $(a_i, b_{i+1}) \in E(G)$.

Now, $u \in C_0 = S(u)$, and since C_0 is an SCC, u can reach a_0 within C_0. Then $a_0 \to b_1$ in G, and b_1 can reach a_1 within C_1 (same SCC). Then $a_1 \to b_2$, etc. Finally, $b_m \in C_m = S(v)$, and since $v \in C_m$, b_m can reach v within C_m. Thus u can reach v in G.

Therefore u can reach v in $G \Leftrightarrow S(u)$ can reach $S(v)$ in $scc(G)$.

problem 2.

1. proof

$\Rightarrow$ If G has an Euler tour, then in-deg(v) = out-deg(v) for all v:

Let T be an Euler tour. T is a closed walk that uses every edge exactly once. Each time T visits a vertex v, it enters via one edge (contributing +1 to in-degree) and leaves via another edge (contributing +1 to out-degree). Since T is a cycle returning to the start vertex s, the start vertex also has equal in-degree and out-degree (the last edge enters s, the first edge leaves s). Since every edge is used exactly once, in-deg(v)=out-deg(v) for all v.

$\Leftarrow$ If in-deg(v) = out-deg(v) for all v, then G has an Euler tour:

Since G is strongly connected and every vertex has equal in-degree and out-degree, start at any vertex and greedily walk along unused edges, whenever we enter a vertex v, we can always leave (because in-deg(v) = out-deg(v), so the number of used in-edges equals the number of used out-edges at any point). This walk must eventually return to the start vertex.

If this closed walk does not cover all edges, then since G is strongly connected, there must be some vertex w on the walk that has unused edges. Start a new walk from w using only unused edges, which again must close back to w. Splice this new cycle into the original walk at w. Repeat until all edges are covered. The result is an Euler tour.

2. ① check precondition: verify in-deg(v) = out-deg(v) for all v. If not, no Euler tour exists.

② Maintain a "current edge" pointer for each vertex (initially pointing to the first edge in the adjacency list). This allows O(1) access to the next unused edge.

③ Start from any vertex s. Use a stack-based approach:

push s onto the stack.

while the stack is not empty:

Let v = top of stack.

If v has an unused outgoing edge (v, w): advance v's current edge pointer, push w onto the stack.

Else: pop v from the stack and prepend v to the result list.

④ The resulting list is the Euler tour.

Time complexity: O(E). Each edge is traversed exactly once (advancing the current edge pointer). Each vertex is pushed/popped from the stack at most O(deg(v)) times. The total work across all vertices is $O(\Sigma \deg(v)) = O(E)$

Problem 3.

Vertices: {A, B, C, D, E, F, G}

Edges: A→B, A→C, B→C, B→D, C→E, D→E, D→F, G→F, G→E

Adjacency list: A→{B, C}, B→{C, D}, C→{E}, D→{E, F}, G→{F, E}.

In-degrees: A: 0, B: 1 (←A), C: 2 (←A, B), D: 1 (←B)
E: 3 (← C, D, G), F: 2 (← D, G)
G: 0

Vertices with in-degree 0: A, G (source vertices).

Sorting 1 : DFS-based, starting from A.

We use DFS-based topological sort. Process vertices in reverse DFS finish order.

DFS from A:

DFS(A): visit B first

DFS(B): visit C first

DFS(C): visit E

DFS(E): no unvisited neighbors, finish E (finish time 1)

finish C (finish time 2)

visit D

DFS(D): E already visited, visit F

DFS(F): no unvisited neighbors, finish F (finish time 3)

finish D (finish time 4)

finish B (finish time 5)

visit C (already visited)

finish A (finish time 6)

G has no been visited. Start DFS from G.

DFS(G): F and E already visited, finish G (finish time 7)

Reverse finish order: G, A, B, D, C, F, E

Verification: every edge goes from earlier to later in this order.

Sorting 2 : DFS-based, starting from G.

DFS(G): visit F first

DFS(F): no unvisited neighbors, finish F (finish time 1)

visit E

DFS(E): no unvisited neighbors, finish E (finish time 2)

finish G (finish time 3)

A not visited. Start DFS from A:

DFS (A) : visit B

DFS (B) : visit C

DFS (C) : E already visited, finish C (finish time 4)

visit D

DFS (D) : E visited, F visited, finish D (finish time 5)

finish B (finish time 6)

visit C (already visited)

finish A (finish time 7)

Reverse finish order : A, B, D, C, G, E, F.

Verification : every edge goes from earlier to later in this order.

Sorting 3 : Kahn's Algorithm (BFS-based).

Using Kahn's algorithm (BFS-based). Process in-degree 0 vertices from a queue, breaking ties alphabetically.

Initial queue (in-degree 0) : {A, G}. Process alphabetically.

Step 1 : Remove A. Reduce in-degree of B (1 → 0), C (2 → 1).
Queue : {B, G}.

Step 2 : Remove B. Reduce in-degree of C (1 → 0), D (1 → 0). Queue {C, D, G}

Step 3 : Remove C. Reduce in-degree of E (3 → 2). Queue : {D, G}

Step 4 : Remove D. Reduce in-degree of E (2 → 1), F (2 → 1). Queue : {G}

Step 5 : Remove G. Reduce in-degree of F (1 → 0), E (1 → 0). Queue : {E, F}

Step 6 : Remove E. Queue : {F}.

Step 7 : Remove F. Queue : {}.

Topological order : A, B, C, D, G, E, F

Verification : every edge goes from earlier to later.

In short : All three orderings are valid topological sorts satisfying the constraint that for every edge $u \to v$, u appears before v. Sortings 1 and 2 use DFS-based topological sort (reverse finish order) with different starting vertices, while Sorting 3 uses Kahn's algorithm (BFS-based). The different orderings arise from different algorithms and tie-breaking choices.